```
                        custom=custom),     #5
    )
    print(recommendation)


# Use asyncio.run to execute the async function
asyncio.run(run_recommendation())    #5

###Output
One movie that fits the criteria of being about time travel, set in a
medieval period, and being a comedy is "The Visitors" (Les Visiteurs)
from 1993. This French film, directed by Jean-Marie Poiré, follows a
knight and his squire who are transported to the modern era by a
wizard’s spell gone wrong.…
```

#1 Defines a prompt with placeholders
#2 Configures a prompt template and input variable definitions
#3 Creates a kernel function from the prompt
#4 Creates an asynchronous function to wrap the function call
#5 Sets the kernel function arguments

Go ahead and debug this example (F5), and wait for the output to be generated. That is the basis for setting up SK and creating and exercising semantic functions. In the next section, we move on to see how a semantic function can be registered as a skill/plugin.

5.4 Synergizing semantic and native functions

Semantic functions encapsulate a prompt/profile and execute through interaction with an LLM. Native functions are the encapsulation of code that may perform anything from scraping websites to searching the web. Both semantic and native functions can register as plugins/skills in the SK kernel.

A function, semantic or native, can be registered as a plugin and used the same way we registered the earlier function directly with our API calls. When a function is registered as a plugin, it becomes accessible to chat or agent interfaces, depending on the use case. The next section looks at how a semantic function is created and registered with the kernel.

5.4.1 Creating and registering a semantic skill/plugin

The VS Code extension for SK provides helpful tools for creating plugins/skills. In this section, we’ll use the SK extension to create a